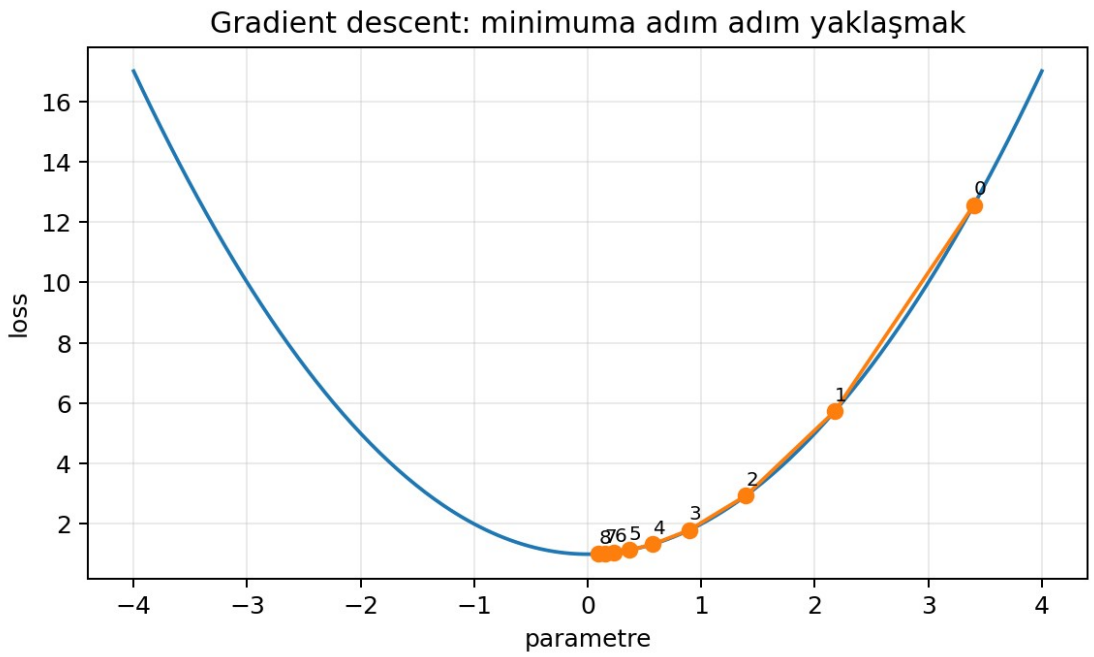
13. CALCULUS & OPTIMIZATION

Veri Bilimci için Ayrıntılı Konu Anlatımı



Bu modülün amacı
Calculus'u üniversite sınavı tarzı işlem ezberine dönüştürmeden; modelin hatasının parametrelere göre nasıl değiştiğini, gradient'in neden yön gösterdiğini ve optimizasyon algoritmalarının modeli nasıl eğittiğini anlamak.

Kazanım	Bu modül sonunda
Türev sezgisi	Bir fonksiyonun lokal değişim hızını okuyabilirsin.
Partial derivative	Birden çok parametreli loss'ta tek parametrenin etkisini ayırabilirsin.
Gradient	En hızlı artış yönünü ve -gradient'in neden iniş yönü olduğunu açıklayabilirsin.
Optimization	Gradient descent, learning rate, momentum ve adaptif optimizer mantığını anlarsın.
ML bağlantısı	Linear/logistic regression ve neural network eğitimindeki optimizasyon sürecini okuyabilirsin.

1. Calculus veri biliminde neyi çözer?

Machine learning modeli bir parametre kümesi taşır: katsayılar, ağırlıklar, bias terimleri veya milyonlarca neural network parametresi. Eğitim sırasında amaç, bu parametreleri loss'u azaltacak şekilde güncellemektir. Calculus bu loss'un parametrelere karşı nasıl değiştiğini söyler; optimization ise bu bilgiyi kullanarak iyi parametreleri arar.

Soru	Calculus/optimization cevabı
Bir parametreyi az değiştirirsem loss ne olur?	Derivative / partial derivative
Tüm parametreleri hangi yönde değiştirmeliyim?	Gradient
Minimuma nasıl yaklaşırım?	Gradient-based optimization
Adımlar ne kadar büyük olmalı?	Learning rate
Neden model bazen yakınsamaz?	Non-convexity, bad scale, LR, noisy gradients
Backpropagation ne yapıyor?	Chain rule ile gradient hesaplıyor

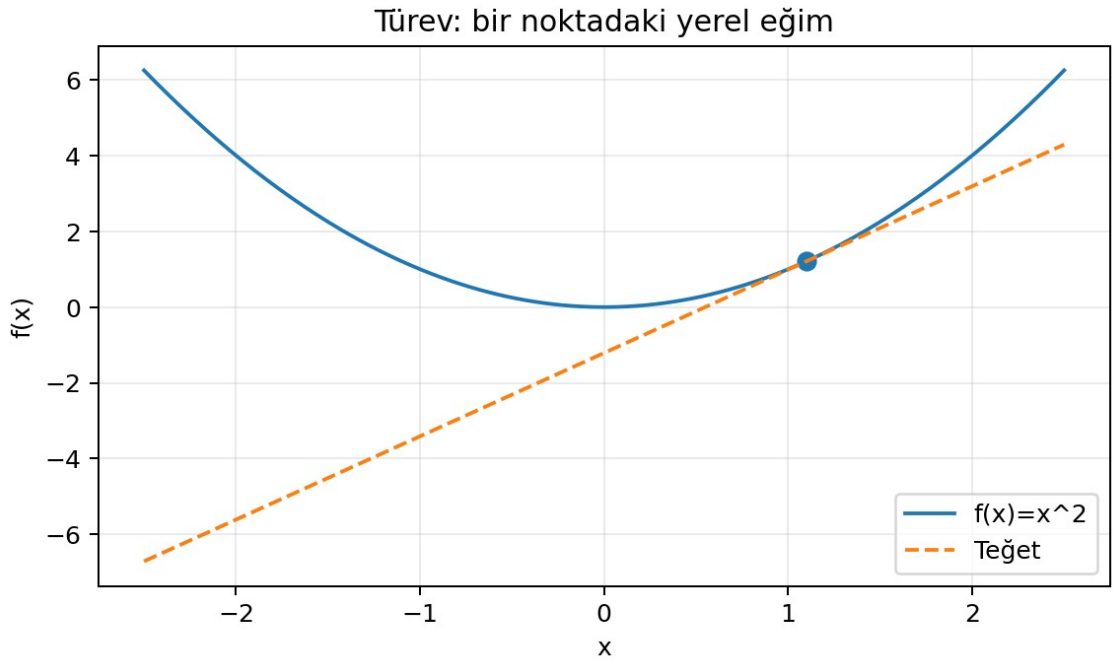
Ana zihinsel model
Model training = loss yüzeyinde iyi parametreleri arama problemi. Gradient bize bulunduğumuz noktadaki en dik yukarı yönü verir; negatif gradient ise iniş yönüdür.

2. Fonksiyon, değişken ve değişim oranı

Bir fonksiyon, girdiyi çıktıya dönüştüren kuraldır. Örneğin f(x)=x^2. ML'de ise loss fonksiyonu L(w), parametre vektörü w'yi bir hata değerine dönüştürür.

Matematik	ML karşılığı
x	Tek parametre veya input
f(x)	Tahmin veya loss
w	Parametre vektörü
L(w)	Model loss'u
min L(w)	Training hedefi

3. Türev: lokal eğim



Türev, x çok küçük miktarda değiştiğinde f(x)'in yaklaşık ne kadar değişeceğini söyler. Geometrik olarak eğriye çizilen teğetin eğimidir.

Yerel lineer yaklaşım
Küçük Δx için f(x+Δx) ≈ f(x) + f'(x)Δx. Gradient descent'in temel sezgisi budur: loss'un küçük parametre değişikliklerine vereceği tepkiyi yaklaşık tahmin ederiz.

İşaret	Yorum
f'(x) > 0	x arttıkça fonksiyon lokal olarak artıyor.
f'(x) < 0	x arttıkça fonksiyon lokal olarak azalıyor.
f'(x) = 0	Düz nokta: minimum, maksimum veya saddle olabilir.

4. Temel türev kuralları

Kural	Sonuç
d/dx c	0
d/dx x^n	n x^(n-1)
d/dx (f+g)	f' + g'
d/dx (c f)	c f'
Product rule	(fg)' = f'g + fg'
Quotient rule	(f/g)' = (f'g - fg') / g^2
Chain rule	d f(g(x))/dx = f'(g(x)) g'(x)

Veri bilimci için öncelik
Power rule ve özellikle chain rule çok önemlidir. Neural network backpropagation, katmanlardan geçen chain rule uygulamasıdır.

5. Chain rule: bileşik fonksiyonların anahtarı

Bir çıktı birçok ara dönüşümden geçiyorsa, en dış fonksiyonun türevi ile iç fonksiyonların türevleri çarpılır. Örneğin z=w x + b ve y=sigmoid(z) ise y'nin w'ye etkisi iki adım üzerinden hesaplanır.

```
# Kavramsal zincir:
w -> z = w*x + b -> y_hat = sigmoid(z) -> loss

# Chain rule:
dL/dw = dL/dy_hat * dy_hat/dz * dz/dw
```

Backpropagation
Backpropagation ayrı bir sihirli algoritma değildir. Computational graph üzerindeki chain rule'un verimli biçimde uygulanmasıdır.

6. Partial derivative

Bir fonksiyon birden fazla değişkene bağlı olduğunda, diğerlerini sabit tutup yalnızca bir değişken yönündeki değişim oranına partial derivative denir.

Örnek: L(w1,w2) = (w1-2)^2 + 3(w2+1)^2

Türev	Sonuç
∂L/∂w1	2(w1-2)
∂L/∂w2	6(w2+1)

Bu iki partial derivative birlikte gradient vektörünü oluşturur. Gerçek modellerde yüzlerce, milyonlarca hatta milyarlarca parametre için aynı fikir kullanılır.

7. Gradient

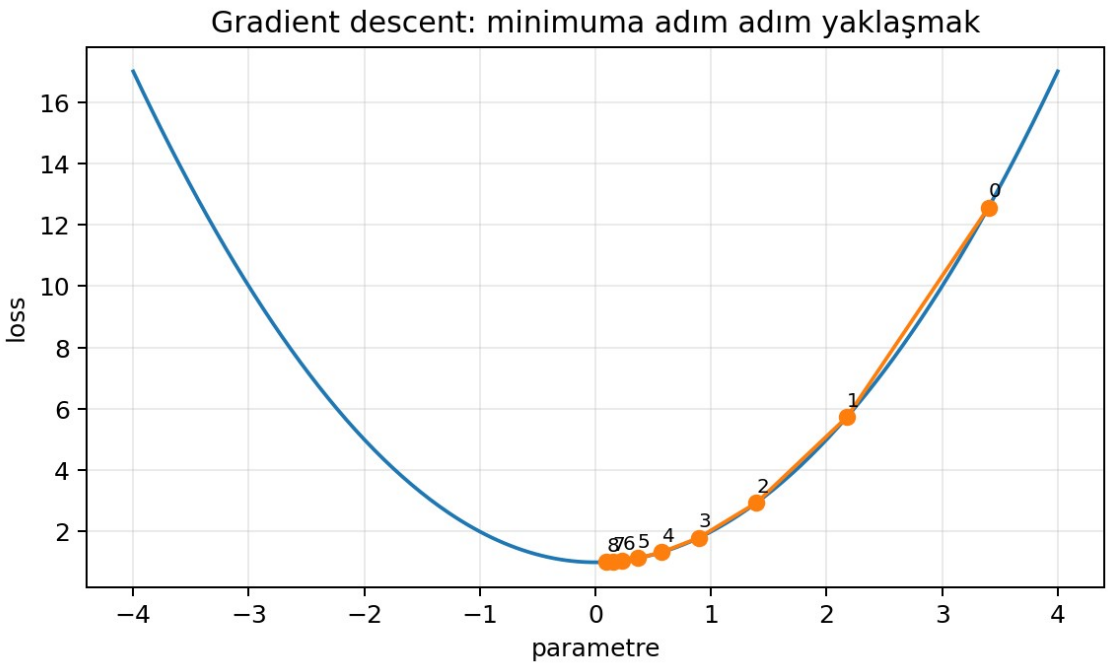
Gradient, bir scalar fonksiyonun tüm parametrelere göre partial derivative'larını bir araya getiren vektördür. ∇L(w), loss'un en hızlı arttığı yönü gösterir.

Neden negatif gradient?
Gradient en hızlı yükseliş yönüyse, minimum ararken ters yöne gideriz: w_new = w_old - learning_rate * gradient.

```
def loss(w1, w2):
    return (w1 - 2)**2 + 3*(w2 + 1)**2

def grad(w1, w2):
    return np.array([
        2*(w1 - 2),
        6*(w2 + 1)
    ])
```

8. Gradient descent



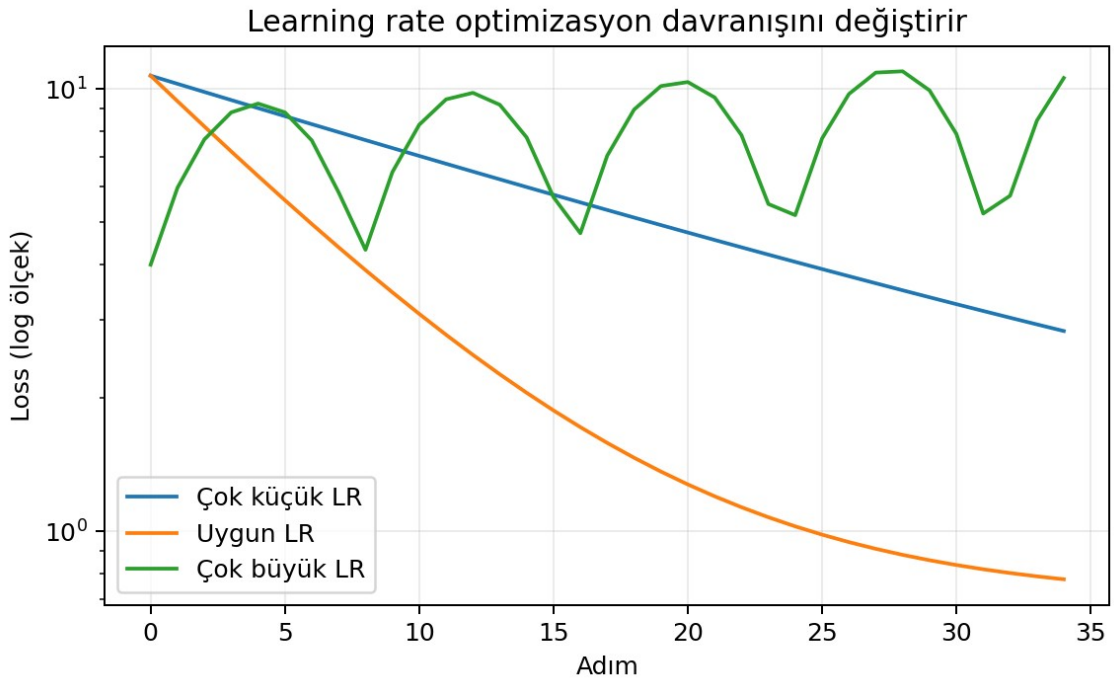
Gradient descent parametreleri iteratif olarak günceller. Her adımda gradient hesaplanır ve parametreler ters yönde hareket ettirilir.

```
w = 3.5
lr = 0.1

for step in range(20):
    gradient = 2*w        # L(w)=w^2 için
    w = w - lr*gradient
    print(step, w, w**2)
```

Terim	Anlamı
step / iteration	Bir parametre güncelleme adımı
learning rate	Gradient'in ne kadarının uygulanacağı
gradient	Lokal eğim bilgisi
convergence	Parametre/loss'un kararlı bölgeye yaklaşması

9. Learning rate



LR durumu	Davranış
Çok küçük	Güvenli ama çok yavaş training
Uygun	Hızlı ve kararlı yakınsama
Çok büyük	Minimumu atlama, oscillation veya divergence

Pratik gerçek
Learning rate çoğu gradient-based modelde en etkili hyperparameter'lardan biridir. Model mimarisi doğru olsa bile kötü LR training'i bozabilir.

10. Batch, stochastic ve mini-batch gradient descent

Yöntem	Gradient hangi veriyle?	Artı / eksi
Batch GD	Tüm training set	Kararlı gradient; büyük veride pahalı
SGD	Tek gözlem	Ucuz/noisy; yüksek varyans
Mini-batch	Küçük batch	Modern deep learning standardı

Mini-batch yaklaşımı hesaplama verimliliği ile gradient kalitesi arasında iyi denge kurar ve GPU paralelliğinden yararlanır.

Epoch vs iteration
1 epoch = training setin tamamının yaklaşık bir kez görülmesi. 1 iteration = bir mini-batch üzerinde bir parametre güncellemesi.

11. Convexity

Convex bir fonksiyonda iki nokta arasındaki doğru parçası fonksiyonun üzerinde kalmaz; sezgisel olarak tek bir kase benzeri global minimum yapısı vardır.

Problem	Optimizasyon sonucu
Convex	Her local minimum global minimumdur.
Strongly convex	Minimum daha iyi tanımlı; yakınsama analizi daha güçlüdür.
Non-convex	Local minima, saddle point ve karmaşık loss yüzeyi olabilir.

ML örneği
Linear regression'ın squared-error problemi uygun koşullarda convex'tir. Deep neural network loss'ları genel olarak non-convex'tir.

12. Local minimum, global minimum ve saddle point

Gradient'in sıfır olması tek başına minimum olduğumuzu garanti etmez. Nokta local maximum veya saddle point olabilir.

- Global minimum: tüm uzaydaki en düşük loss.
- Local minimum: yakın çevrede en düşük loss.
- Saddle point: bazı yönlerde yükselip bazı yönlerde düşen düz nokta.

Deep learning sezgisi
Yüksek boyutlu loss yüzeylerinde saddle point'ler klasik 2D local minimum resminden daha önemlidir.

13. İkinci türev ve curvature

İkinci türev eğimin ne kadar hızlı değiştiğini, yani curvature'ı anlatır. Tek değişkende f''(x)>0 yerel olarak yukarı açılan, f''(x)<0 aşağı açılan yapı anlamına gelir.

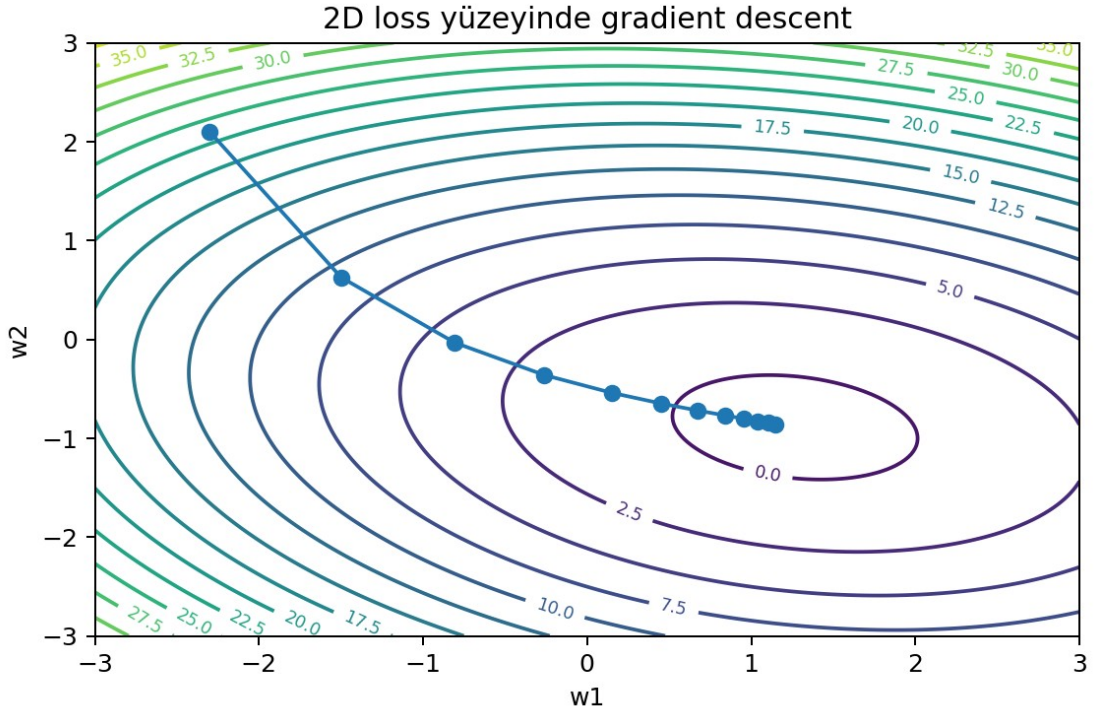
Durum	Sezgi
f'(x*) = 0, f''(x*) > 0	Local minimum adayı
f'(x*) = 0, f''(x*) < 0	Local maximum adayı
f''(x*) ≈ 0	Daha dikkatli analiz gerekir

14. Hessian matrix

Çok değişkenli fonksiyonlarda ikinci partial derivative'lar Hessian matrix içinde toplanır. Hessian loss yüzeyinin curvature bilgisini verir.

Neden her zaman kullanmıyoruz?
n parametre için Hessian n x n boyutundadır. Milyonlarca parametreli neural network'te tam Hessian hesaplamak ve saklamak pratik değildir. Bu nedenle first-order optimizer'lar yaygındır.

15. Optimization landscape



Contour plot aynı loss seviyelerini gösterir. Gradient contour çizgilerine diktir ve en hızlı artış yönünü gösterir; negatif gradient minimuma doğru yönlenir.

Feature scaling bağlantısı
Feature ölçekleri çok farklıysa loss yüzeyi uzun ve dar elipsler gibi olabilir. Bu durum gradient descent'in zikzak çizmesine ve yavaşlamasına neden olabilir.

16. Momentum

Momentum yalnızca o anki gradient'i değil önceki güncellemelerin yönünü de taşır. Böylece tutarlı yönde hızlanıp dar vadilerdeki zikzağı azaltabilir.

```
v = beta * v + (1 - beta) * grad
w = w - lr * v
```

Etki	Momentum sezgisi
Tutarlı gradient yönü	Hız birikir
Gradient yönü sık değişiyor	Salınım yumuşatılır
Dar valley	İlerleme hızlanabilir

17. RMSProp ve Adam

Adaptif optimizer'lar her parametre için gradient geçmişine göre farklı efektif adım büyüklükleri oluşturur.

Optimizer	Ana fikir
SGD	Tek global learning rate
Momentum SGD	Yön geçmişini taşır
RMSProp	Squared gradient ortalamasıyla parametre başına ölçekleme
Adam	Momentum + adaptive scaling

Adam her zaman en iyi mi?
Hayır. Adam hızlı ve kullanışlı bir default olabilir; ancak bazı görevlerde iyi ayarlanmış SGD/Momentum daha iyi generalization verebilir. Optimizer seçimi deneysel bir karardır.

18. Learning-rate schedules

Schedule	Mantık
Step decay	Belirli epoch'larda LR azalt
Exponential decay	LR'ı sürekli çarpanla küçült
Cosine decay	Yumuşak cosine eğrisiyle azalt
Warmup	Training başında küçük LR'dan başlat
One-cycle	Önce artır sonra düşür

Training'in başında daha büyük adımlar hızlı ilerlemeyi, sonlarında küçük adımlar hassas ayarı kolaylaştırabilir.

19. Loss fonksiyonları ve türev davranışı

Loss	Tipik kullanım	Optimizasyon özelliği
MSE	Regression	Büyük hataları karesel cezalandırır; smooth
MAE	Robust regression	Outlier'a daha dayanıklı; 0'da non-differentiable
Binary cross-entropy	Binary classification	Probability modelleme ile uyumlu
Cross-entropy	Multiclass classification	Softmax ile yaygın
Huber	Robust regression	MSE ve MAE arasında geçiş

Loss != business metric
Model MSE ile optimize edilip MAE ile raporlanabilir; classifier cross-entropy ile eğitilip F1/AUC ile değerlendirilebilir. Training objective ile business/evaluation metric aynı olmak zorunda değildir.

20. Linear regression gradient'i

Tahmin y_hat = Xw, loss L(w)=1/n ||Xw-y||^2 ise gradient aşağıdaki yapıdadır:

```
gradient = (2 / n) * X.T @ (X @ w - y)
w = w - lr * gradient
```

Burada lineer cebir ve calculus birleşir: Xw tahmin, residual = Xw-y hata, X.T ise bu hatanın her katsayıya etkisini geri dağıtır.

21. Logistic regression optimizasyonu

Logistic regression, sigmoid ile probability üretir ve çoğunlukla binary cross-entropy minimize eder. Closed-form çözüm yoktur; gradient-based veya ikinci-order yöntemler kullanılır.

```
z = X @ w
p = 1 / (1 + np.exp(-z))

gradient = (1 / n) * X.T @ (p - y)
w = w - lr * gradient
```

Önemli bağlantı
Linear regression'da squared error, logistic regression'da log-loss; ama eğitim mantığı aynıdır: loss -> gradient -> parameter update.

22. Regularization optimizasyonu nasıl değiştirir?

Regularization loss'a parametre büyüklüğünü cezalandıran ek terim ekler. Böylece optimizasyon yalnızca training error'u değil model karmaşıklığını da dengeler.

Regularization	Ek terim	Gradient etkisi
L2 / Ridge	lambda ||w||_2^2	Gradient'e 2 lambda w eklenir
L1 / Lasso	lambda ||w||_1	0 noktasında klasik türev yok; subgradient/proximal yöntemler kullanılabilir

Geometrik yorum
L2 ağırlıkları sürekli küçültür; L1'in geometrisi bazı katsayıların tam 0 olmasını teşvik eder.

23. Differentiability ve subgradient

Her yararlı loss her noktada türevlenebilir olmak zorunda değildir. |x| fonksiyonu x=0'da klasik türeve sahip değildir ama subgradient yaklaşımıyla optimize edilebilir.

- ReLU x=0'da klasik anlamda differentiable değildir.
- MAE residual=0 noktasında differentiable değildir.
- Pratik ML framework'leri bu noktalarda uygun convention/subgradient kullanır.

24. Automatic differentiation

PyTorch, TensorFlow ve JAX gibi framework'ler computational graph üzerinden türevleri otomatik hesaplar. Bu, symbolic algebra'dan farklı olarak gerçek yapılan işlemlerin türevlerini zincirler.

```
import torch

w = torch.tensor(3.0, requires_grad=True)
loss = (w - 2)**2

loss.backward()

print(w.grad)   # d(loss)/dw
```

Bilmen gereken
Framework gradient'i otomatik hesaplıyor diye calculus gereksiz olmaz. Gradient'in neyi temsil ettiğini bilmek; exploding/vanishing gradients, detach, optimizer davranışı ve debugging için kritiktir.

25. Vanishing ve exploding gradients

Derin computational graph'larda chain rule çok sayıda türevin çarpılmasına yol açar. Çarpanlar sürekli 1'den küçükse gradient küçülebilir, büyükse patlayabilir.

Problem	Belirti	Yaygın araçlar
Vanishing gradient	İlk katmanlar çok yavaş öğrenir	ReLU türev ailesi, normalization, residual connections
Exploding gradient	Loss/weights NaN veya aşırı büyük olur	Gradient clipping, initialization, normalization

26. Optimization ve generalization aynı şey değildir

Training loss'u minimum yapmak training verisine uyumu artırır; ama asıl amaç unseen data performansıdır. Optimization başarısı ile generalization başarısını ayırmak gerekir.

Optimization sorusu	Generalization sorusu
Training loss düşüyor mu?	Validation performansı iyi mi?
Gradient sağlıklı mı?	Overfitting var mı?
Optimizer yakınsıyor mu?	Model yeni veriye taşınıyor mu?
LR uygun mu?	Regularization / data yeterli mi?

Erken durdurma
Validation loss kötüleşmeye başlarken training loss düşmeye devam ediyorsa early stopping, optimizasyonu generalization hedefiyle sınırlar.

27. Optimization debugging checklist

- Loss başlangıçta beklenen ölçekte mi?
- Gradient'ler sıfır, NaN veya aşırı büyük mü?
- Learning rate aşırı küçük/büyük olabilir mi?
- Feature scale'leri birbirinden çok farklı mı?
- Batch size çok küçük veya çok büyük mı?
- Initialization uygun mu?
- Train/validation ayrımı ve leakage kontrol edildi mi?
- Loss ile task uyumlu mu?
- Regularization aşırı güçlü mü?
- Model kapasitesi problemi çözmeye yetiyor mu?

28. Gerçek vaka: churn modeli optimizasyonu

Bir abonelik şirketinde logistic regression ile churn tahmini yaptığını düşün. Ama model loss'u yavaş düşüyor ve validation AUC sınırlı.

1. Feature'ların scale dağılımlarını kontrol et.
2. StandardScaler uygula ve tekrar eğit.
3. Learning rate'i log scale üzerinde dene: 1e-4, 1e-3, 1e-2, 1e-1.
4. Training ve validation loss curve'lerini birlikte izle.
5. L2 regularization strength'i tune et.
6. Class imbalance varsa loss weighting veya sampling stratejisini değerlendir.
7. Probability calibration ve threshold seçimini optimizasyon sürecinden ayrı değerlendir.

Beklenen düşünce biçimi
Sorun sadece 'optimizer değiştir' değildir. Scale, objective, data distribution, regularization ve evaluation metric birlikte değerlendirilir.

29. NumPy ile mini gradient descent projesi

```
import numpy as np

rng = np.random.default_rng(42)

X = rng.normal(size=(500, 3))
true_w = np.array([2.0, -1.5, 0.7])
y = X @ true_w + rng.normal(scale=0.5, size=500)

w = np.zeros(3)
lr = 0.05

for epoch in range(300):
    pred = X @ w
    error = pred - y
    loss = np.mean(error**2)

    grad = (2 / len(X)) * X.T @ error
    w -= lr * grad

print("w:", w)
print("loss:", loss)
```

Proje geliştirme
Aynı kodda farklı learning rate'leri karşılaştır, loss curve çiz, feature'ları farklı scale'lere getirip convergence farkını gözlemle ve sklearn LinearRegression sonucu ile katsayıları kıyasla.

30. Mülakat soruları

Soru	Kısa cevap
Derivative nedir?	Bir fonksiyonun belirli bir noktadaki lokal değişim hızıdır.
Gradient nedir?	Çok değişkenli scalar fonksiyonun tüm partial derivative'larını içeren vektördür.
Neden -gradient yönüne gideriz?	Gradient en hızlı artış yönünü gösterdiği için ters yön lokal olarak loss'u azaltır.
Learning rate çok büyükse ne olur?	Minimumu aşma, oscillation veya divergence görülebilir.
Mini-batch GD neden yaygın?	Hesaplama verimliliği, GPU paralelliği ve kabul edilebilir gradient gürültüsü dengesi sağlar.
Convex problem neden kolaydır?	Local minimum global minimumdur; optimizasyon garantileri daha güçlüdür.
Chain rule neden deep learning için kritik?	Backpropagation katmanlar arasındaki türevleri chain rule ile birleştirir.
Momentum ne sağlar?	Tutarlı yönde hız biriktirir ve oscillation'ı azaltabilir.
Adam neyi birleştirir?	Momentum benzeri first-moment estimate ile squared-gradient tabanlı adaptif scaling'i.
Loss ve metric farkı nedir?	Loss training için optimize edilen differentiable objective; metric değerlendirme/business amacı için kullanılabilir.
Vanishing gradient nedir?	Chain rule sonucu gradient'lerin derin katmanlarda çok küçülmesi.
Gradient 0 ise mutlaka minimum mu?	Hayır; maximum veya saddle point olabilir.
Hessian nedir?	İkinci partial derivative'lardan oluşan curvature matrisi.
Neden feature scaling GD'yi etkiler?	Loss yüzeyinin geometrisini ve parametre yönlerindeki efektif adım büyüklüğünü değiştirir.

31. Alıştırmalar

8. f(x)=3x^2-4x+2 fonksiyonunun türevini bul ve x=2'deki eğimi yorumla.
9. L(w1,w2)=(w1-3)^2 + 2(w2+2)^2 için gradient'i çıkar.
10. Aynı quadratic loss için learning rate 0.01, 0.1 ve 1.0 ile gradient descent çalıştır; davranışı karşılaştır.
11. Linear regression gradient'ini NumPy ile elle hesapla ve finite difference ile doğrula.
12. StandardScaler öncesi ve sonrası gradient descent convergence'ını karşılaştır.
13. Mini-batch size 1, 32 ve full batch için loss curve'leri çiz.
14. Momentum ekleyerek aynı problemi tekrar çöz ve iteration sayısını karşılaştır.
15. PyTorch autograd ile elle çıkardığın gradient'i doğrula.

32. Final cheat sheet

Kavram	Hatırlanacak fikir
Derivative	Tek yöndeki lokal değişim hızı
Partial derivative	Bir parametrenin etkisi
Gradient	Tüm partial derivative'ların vektörü
Chain rule	Bileşik fonksiyonlarda türevlerin zinciri
Gradient descent	w <- w - lr * grad
Learning rate	Adım büyüklüğü
Mini-batch	Modern training'in pratik dengesi
Convexity	Global minimum yapısını kolaylaştırır
Momentum	Gradient geçmişinden hız taşır
Adam	Momentum + adaptif ölçekleme

Hessian	Curvature / ikinci-order bilgi
Autograd	Computational graph üzerinden otomatik türev
Regularization	Objective'e model karmaşıklığı cezası ekler
Early stopping	Optimization ile generalization arasında fren

Bu modülden sonra
Machine Learning Temelleri, Linear/Logistic Regression, Neural Networks ve Deep Learning eğitim süreçlerindeki loss-gradient-optimizer zincirini formül ezberi yerine mekanizma olarak okuyabilmelisin.